June 18th,2024

Timer Input Assignment Report

Embedded System 3, LAB 05

Document Version 1.0

Edited by:
Farros Ramzy (3767353)

Abstract

This technical report describes about the use of timer bit registers in the STM32 board to write an output and read an input on the STM32 Nucleo board. This report focused on the procedure, context, and knowledge about the timers output registers and PWM pins of the STM32-F303RE board microcontroller.

List of Figures

Table of Contents

1. Introduction

This assignment is meant to understand the concept of the timer input and how the PWM mode of the STM32 Nucleo works. The assignment is starting from the way of configuring the timer system, the alternate functions of the GPIO pin registers, setting up the PWM output, then read the PWM input.

The document will explain each part in the Procedure section, the result will be placed in the Conclusion section, and at least, a recommendation will be made in the Recommendation section. Some references that have been used for the practicum and included in this document are listed in the References part by the end of this report.

1.1. Equipment

The equipment needed to do this research practicum are:

1. A programming platform for the STM32 board (STM32CubeIDE)
2. An STM32-F303RE board
3. One HC-SR04 (Ultrasonic Sensor)
4. A breadboard
5. A bunch of wires

2. Procedure

2.1. Preparation

To start the assignment, it is required to read the GPIO registers in the reference manual of the STM32-F303RE board which is listed in this References section.

After that, to understands the pin position that will be used from this STM32 board, the Nucleo User Manual document must be read.

From that manual, it can be known that the STM32-F303RE pins that can be used are presented in this Figure 2. 1. 1 below.

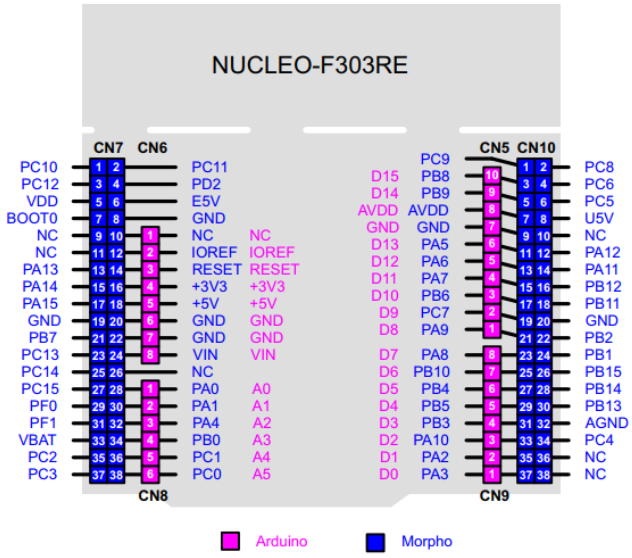

Figure 2. 1. 1 STM32-F303RE Board pinout.

From this Figure 2. 1. 1, it is visible that the pink or purple pins from the board is usable for the Arduino programming.

However, in this practicum, the pins that can be used for the STM GPIO register manipulation are the Morpho pins, reserved for the Nucleo STM32 board.

Nevertheless, there are some pins that cannot be used for external interference such as the internal LED pin, PA5 or PB13, and the internal button, PC13 because these pins are internally built in for the board.

After decided the output pin, the ultrasonic sensor and some jumper wires must be prepared in such a way that shown in this Figure 2. 1. 2 below.

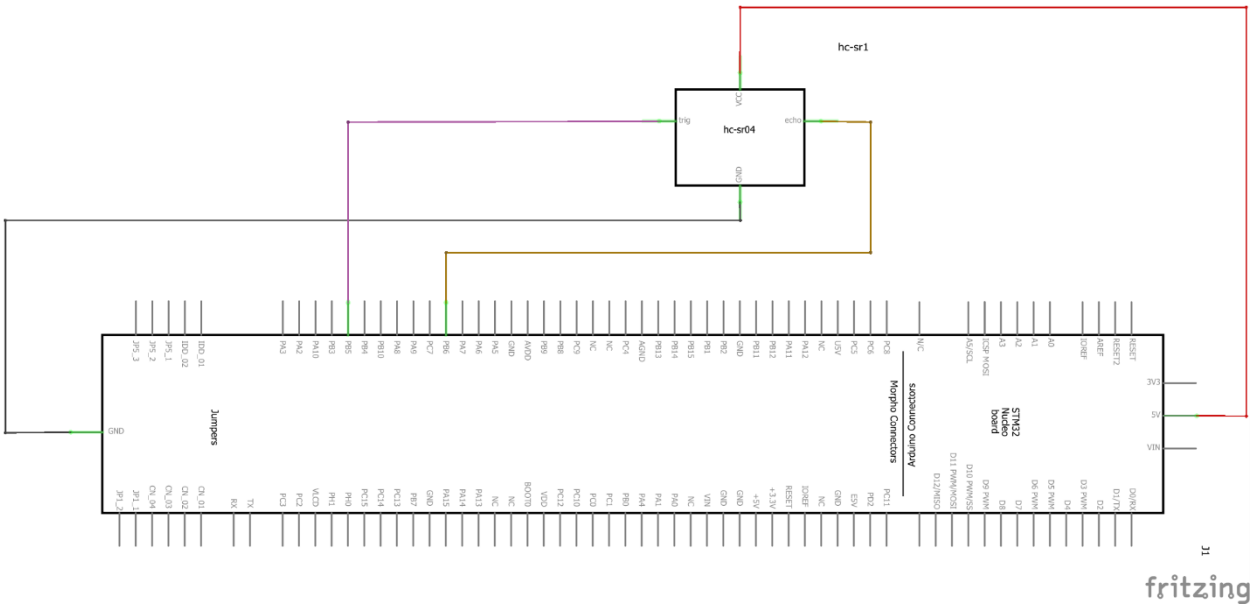

Figure 2. 1. 2 System wiring diagram.

After that, it is recommended that to read the Nucleo F303RE reference manual and the datasheet to locate the PWM pin registers on the microcontroller, then figuring out on how to use them.

If everything is prepared, then it is time to prepare the clock configuration of the system.

This clock configuration can be done in the (ioc) file of the STM project IDE, shown in this Figure 2. 1. 3 below.

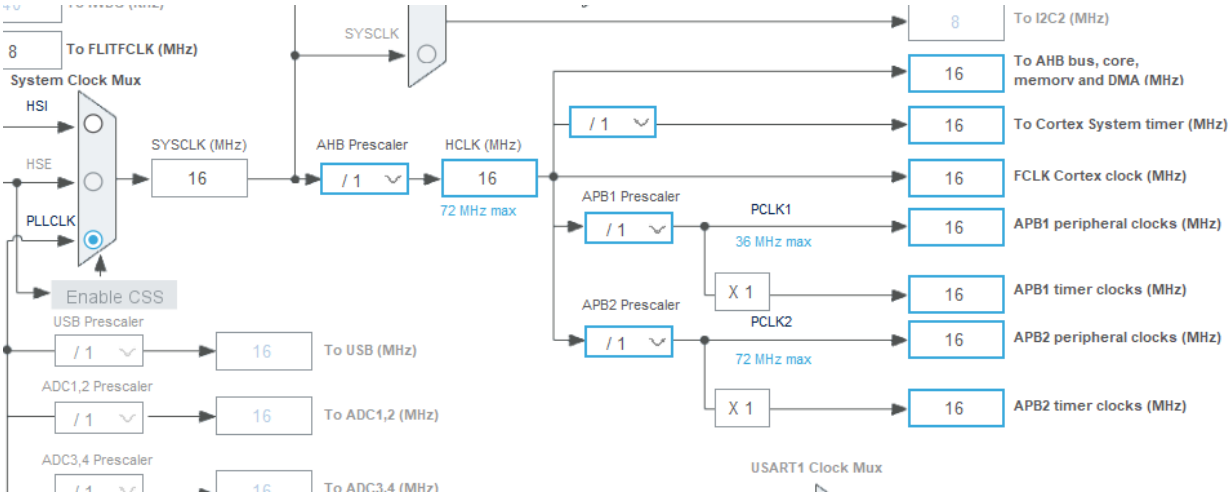

Figure 2. 1. 3 System Clock Configuration.

In this Figure 2. 1. 3 above the system clock is setup to 16 MHz, following to the assignment requirement, and the AHB clock set with no division, since the system will only use the peripheral bus with the timer interrupts to controls the output pins of the system.

2.2. Part A, Setting Up Trigger Output

There are some important steps to do the PWM setup for an output pin register. And in this case, HC-SR04 ultrasonic sensor uses the trigger pin as its PWM output.

As can be seen in the Figure 2. 1. 2 above, the trigger pin is connected to PB5, which is the pin 5 of GPIOB.

The alternate function register of the pinout can be setup in the MODER register as 0b10, shown in this Figure 2. 2. 1 below.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y+1:2y **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O mode.
00: Input mode (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

Figure 2. 2. 1 GPIO MODER guide table.

And in the code, it will be applied like this Figure 2. 2. 2 below.

```
431     //resets the sensor's pin MODE. (PB5)
432     GPIOB->MODER &= ~GPIO_MODER_MODER5;
433     //resets the sensor's pin SPEED.
434     GPIOB->OSPEEDR &= ~GPIO_OSPEEDER_OSPEEDR5;
435     //resets the sensor's PUPD register.
436     GPIOB->PUPDR &= ~GPIO_PUPDR_PUPDR5;
437     //sets the sensor's pin MODE to 0b10 as an alternated output.
438     GPIOB->MODER |= GPIO_MODER_MODER5_1;
439     //sets the sensor's pin SPEED to medium.
440     GPIOB->OSPEEDR |= GPIO_OSPEEDER_OSPEEDR5_0;
441     //resets the PUPD register
442     GPIOB->PUPDR &= ~GPIO_PUPDR_PUPDR5;
```

Figure 2. 2. 2 GPIO setup for trigger pin.

After the MODER, SPEED, and Pull-up/Pull-down resistor are correctly setup, it is time to look at the register map in the STM32 Nucleo F303RE Datasheet to find the correct Alternate Function

register that must be fit with pinout 5 of Port B GPIO. This Alternate Function also must connect to one of the timer registers of the Nucleo, including its port Channel.

And in this case, **AF2** (Alternate Function 2) is happened to be the pin map along these registers, which connect **PB5** to **Timer 3** in **Channel 2** which is on the low register of the alternate function of the Nucleo board.

Now, looking back at the reference manual, the AF2 is included in the low register of the Alternate Function (AFRL) like what is shown in this Figure 2. 2. 3 below.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR7[3:0]				AFR6[3:0]				AFR5[3:0]				AFR4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR3[3:0]				AFR2[3:0]				AFR1[3:0]				AFR0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRy[3:0]:** Alternate function selection for port x pin y (y = 0..7)
These bits are written by software to configure alternate function I/Os

AFRy selection:
0000: AF0
0001: AF1
0010: AF2
0011: AF3
0100: AF4
0101: AF5
0110: AF6
0111: AF7
1000: AF8
1001: AF9
1010: AF10
1011: AF11
1100: AF12
1101: AF13
1110: AF14
1111: AF15

Figure 2. 2. 3 Alternate Function Register Low guide table.

Therefore, the setup of the pinout should use the AFRL (AFR[0] in code mapping) with the 0b0010 as its value like in this Figure 2. 2. 4 below.

```
443     //resets the high alternate function for pin PB5
444     GPIOB->AFR[0] &= ~GPIO_AFRL_AFRL5;
445     //activate alternate function 2 (AF2) to use timer 3 on channel 2 PB5
446     GPIOB->AFR[0] |= 0b0010 << GPIO_AFRL_AFRL5_Pos;
447 }
```

Figure 2. 2. 4 AFRL setup for trigger pin.

After the pin is correctly configured, now is the time to setup the related timer and the channel of the GPIO, to control the PWM value.

To do this, the first thing needed is to enable the Timer 3 in APB Register. And as can be seen in this Figure 2. 2. 5 below, Timer 3 is placed on pin 1 of the APB Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	I2C3 RST(1)	DAC1 RST	PWR RST	Res	DAC2RST	CAN RST	Res	USB RST	I2C2 RST	I2C1 RST	UART5 RST	UART4 RST	USART3 RST	USART2 RST	Res
		rw	rw		rw	rw		rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 RST	SPI2 RST	Res	Res	WWDG RST	Res	Res	Res	Res	Res	TIM7 RST	TIM6 RST	Res	TIM4 RST	TIM3 RST	TIM2 RST
rw	rw			rw						rw	rw		rw	rw	rw

Figure 2. 2. 5 APB1 RCC guide table.

This makes the setup of the register clock control must be 0b0010 in binary, or value 1 << 1.

After that, the timer must be put on reset first by using the clear bit on the timer enabler (pin CR1) to make sure that the system will not use the other setup that might be placed on the memory before this practicum.

And then, since the system uses 16 MHz, shown in this Figure 2. 1. 3 above, the pre-scaler and the auto reload register must be calculated to use the 16 MHz per reload period. And since the assignment required the 65 milliseconds pulsing for the trigger, a setup for Timer 3 can be written such as this Figure 2. 2. 6 below.

```
331     //Pre-scale the system using 16MHz frequency.
332     TIM3->PSC = 16 - 1;
333     //set the PWM period to 65ms.
334     TIM3->ARR = 65000 - 1;
```

Figure 2. 2. 6 Timer 3 pre-scaler and ARR setup.

After that, the capture compare of the Timer 3 can be set to 10 microseconds for the routine trigger pulse, and enables the update event with EGR register, such is shown in this Figure 2. 2. 7 below.

```
335     //set up the capture compare register to 10 microsecond pulse.
336     TIM3->CCR2 = 10;
337     // Generate an update event to load the prescaler and ARR values
338     TIM3->EGR |= TIM_EGR_UG;
```

Figure 2. 2. 7 capture compare and event updater setup.

Once the Timer 3 has been enabled, the channel of the Timer register can be setup properly.

There are three things that must be done on the channel setup for the output PWM pin register of the STM32 Nucleo board.

The first one is to enable the capture compare register pin related to the channel, clean bits the channel mode of the pins, reset the interrupt since it is not being used in this first part of the assignment, and enabling the preload.

This method can be seen in this Figure 2. 2. 8 below.

```
347 void Channel2_Init(void) {
348     // Clear output compare mode bits for channel 2
349     TIM3->CCMR1 &= ~TIM_CCMR1_OC2M;
350     // PWM mode 1
351     TIM3->CCMR1 |= (TIM_CCMR1_OC2M_1 | TIM_CCMR1_OC2M_2);
352     // Enable preload for CCR2 to allow CCR2 to be loaded with a new value
353     TIM3->CCMR1 |= TIM_CCMR1_OC2PE;
354     // Clear polarity bit to make output active high
355     TIM3->CCER &= ~TIM_CCER_CC2P;
356     // Enable output for CH2
357     TIM3->CCER |= TIM_CCER_CC2E;
358 }
```

Figure 2. 2. 8 Timer 3 Channel 2 configuration.

And with that, the trigger setup is done.

2.3. Part B, Setting Up Echo Input

Looking at the Figure 2. 1. 2 above, the Echo Input pin is connected to the PB6, which is the GPIO port B pin 6.

Looking at the Datasheet, the pin is also connected to AF2, but in Channel 1 and Timer 4. This will make the input setup for PB6 looks like this Figure 2. 3. 1 below.

```
407 void Echo_Init(void) {
408     //resets the sensor's pin MODE. (PB6)
409     GPIOB->MODER &= ~GPIO_MODER_MODER6;
410     //resets the sensor's pin SPEED.
411     GPIOB->OSPEEDR &= ~GPIO_OSPEEDER_OSPEEDR6;
412     //resets the sensor's PUPD register.
413     GPIOB->PUPDR &= ~GPIO_PUPDR_PUPDR6;
414     //sets the sensor's pin MODE to 0b10 as an alternated input.
415     GPIOB->MODER |= GPIO_MODER_MODER6_1;
416     //sets the sensor's pin SPEED to medium.
417     GPIOB->OSPEEDR |= GPIO_OSPEEDER_OSPEEDR6_0;
418     //set PUPD to pull-down.
419     GPIOB->PUPDR |= GPIO_PUPDR_PUPDR6_0;
420     //resets the high alternate function for pin PB6.
421     GPIOB->AFR[0] &= ~GPIO_AFRL_AFRL6;
422     //activate alternate function 2 (AF2) to use timer 4 on channel 1 PB6
423     GPIOB->AFR[0] |= 0b0010 << GPIO_AFRL_AFRL6_Pos;
424 }
```

Figure 2. 3. 1 Echo pin in GPIOB 6 configuration.

And while the trigger pulsed per timer, the echo should read the input as interrupt. And because the system should read the echo any time during the period tick to evade any lost input, the ARR

of the Timer 4 must be setup to the max value. This will make the setup of Timer 4 looks like this Figure 2. 3. 2 below.

```
364 void Timer4_Init(void) {
365     // Enable clock for TIM4
366     RCC->APB1ENR |= RCC_APB1ENR_TIM4EN;
367     //set the prescaler
368     TIM4->PSC = 16 - 1;
369     //set the auto-reload value to maximum
370     TIM4->ARR = 0xFFFF;
371 
372     //enable the IRQ interrupt of the TIM4 for capturing value on CH1
373     NVIC_EnableIRQ(TIM4_IRQn);
374     NVIC_SetPriority(TIM4_IRQn, 0);
375     //start TIM4
376     TIM4->CR1 |= TIM_CR1_CEN;
377 }
```

Figure 2. 3. 2 Timer 4 configuration with interrupts.

And because the trigger should pulse regularly within a certain time, SYSTICK configuration must be useful as a millis() counter for it.

After that, to read the echo, a UART print must be setup.

And because the system should mention the distance between the sensor to the reflected obstacle in centimeters, the pulse result must be divided into 58.

This will be used in the main while loop, depicted in this Figure 2. 3. 3 below.

```
133     while (1) {
134         /* USER CODE END WHILE */
135         if (millis() - start_millis >= PRINT_DELAY) {
136             if (read_state == 1) {
137                 distance_cm = sensor_pulse / TIME_DIVIDER;
138                 printValue(distance_cm);
139                 read_state = 0;
140             }
141             start_millis = millis();
142         }
143         /* USER CODE BEGIN 3 */
144     }
```

Figure 2. 3. 3 Main while loop implementation with distance division scaler.

The millis() will use the SYSTIC to send the sensor data in UART periodically, and the TIME_DIVIDER is the value to divide the sensor_pulse value to the correct distance scale (centimeters).

3. Experiment Result

After running the system, the echo that received from the sensor can be viewed on the serial monitor such as shown in this Figure 3. 1 below since the UART communication pin has used.

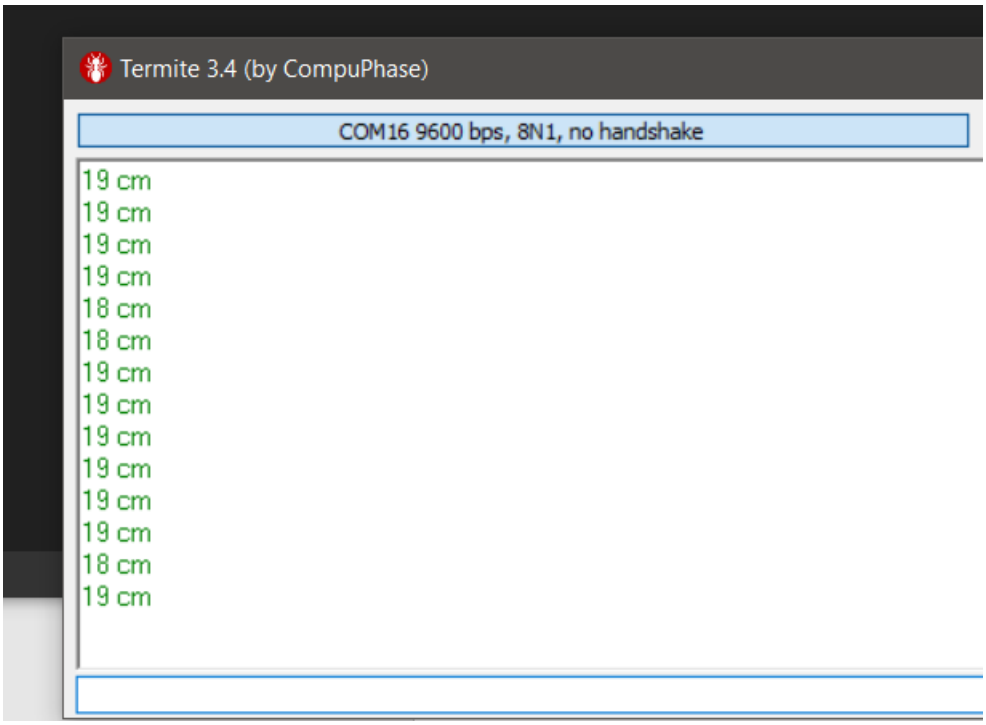

Figure 3. 1 UART results on TRIGGER & ECHO testing.

And looking on the printed distance while testing on it, the distance is pretty accurate since the divider value is being used properly.

4. Conclusion

While doing this assignment and some testing, it is known that using the GPIO timers is much more useful than depending the timer with millis() counter or delay. The system can use the timer register to spot a moment when it must give an exact amount of output within a period without needing any delay function. And from this program, the output result of the system can be figured out numerically, without depending on digital value. The system is basically capable to do analog output while using the Timer and alternate function registers on the Nucleo board.

5. Recommendation

It is recommended to use different channel and Timer per pin in each used GPIO port in case the user need to use multiple tools to act as a timer output because if the system use multiple pins in one Timer and channel for multiple tools for different purposes, the timer would not read the interrupt flag and reload period correctly which can make the system would not be functioned correctly.

And it is also recommended to use a flat surface to test the sensor since the sensor will read a sound reflection. And if the reflective surface is not flat and smooth, the sound will disperse until it is becoming very close to the sensor set point.

6. References

Along this practicum, such documents and references are being used.

- Es_03_lab_03 report, June 2024
- Cortex- M4 Devices Generic User Guide (ARM), 2010
- DM00043574-ReferenceManual_STM32F303RE(www.st.com), January 2017
- UM1724 USER manual STM32 Nucleo-64 boards (MB1136) (www.st.com), April 2019
- Stm32f303re_Datasheet (DocID026415 Rev 5) (www.st.com), October 2016
- Arduino Pull-up Pull-down Resistor (https://arduinogetstarted.com/faq/arduino-pull-up-pull-down-resistor?utm_content=cmp-true)
- Resistor Color Code Calculator (https://www.digikey.com/en/resources/conversion-calculators/conversion-calculator-resistor-color-code)

Git link to access the source file of the assignment:

- Timers output:

https://git.fhict.nl/I424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/3-timers